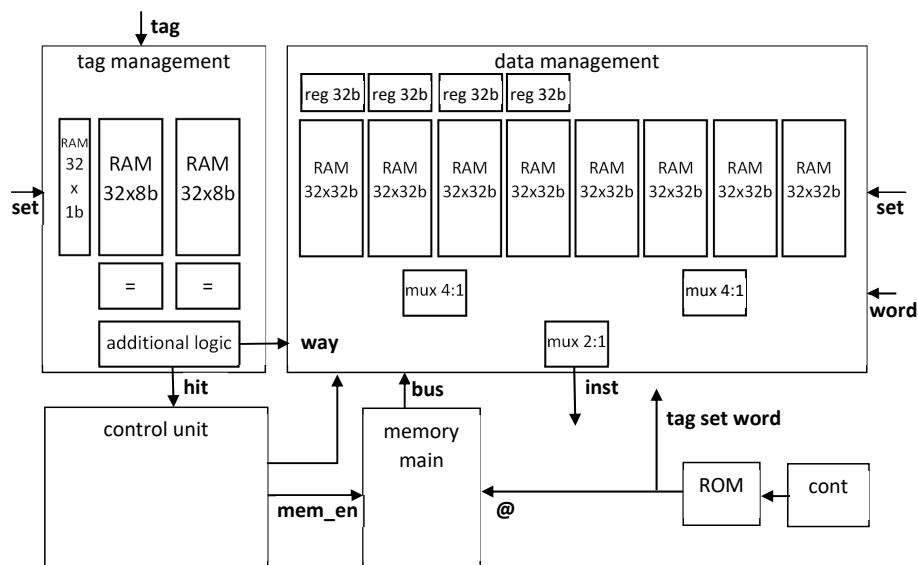


DESIGN OF AN INSTRUCTION CACHE MEMORY

Lab Assignment #4



Computer Architecture and Organization 2
Grado en Ing. Informática – 2º Curso

Luis M. Ramos Alejandro Valero José Luis Briz Javier Resano
luisma@unizar.es alvabre@unizar.es alvabre@unizar.es briz@unizar.es briz@unizar.es
jresano@unizar.es jresano@unizar.es



Escuela de
Ingeniería y Arquitectura
Universidad Zaragoza



Departamento de
Informática e Ingeniería
de Sistemas
Universidad Zaragoza

1 SUMMARY

In this assignment you will design an instruction cache memory. You will test the cache using an address generator. The cache memory must check if each address is a hit or a miss. If miss, the main memory will send the corresponding block.

The goal is to understand how the address is decomposed to obtain tag, set and word, and how does the management of misses work.

*Take a seat at the lab and place your pre-lab work (Sec. 2.1.) on the table, so that the instructor can see it. Before starting to work, log in to Moodle from one of the lab computers and **check in at the attendance control of the session**.*

*The practice is considered completed when the cache is working properly and you have submitted the modified **desing** plus **results of section 3** through Moodle.*

2 SPECIFICATION OF THE CACHE MEMORY

2.1 PRE-LAB: DESIGN ON PAPER

Parameters of the instruction cache to be designed:

- size of 1 KB
- associative with 2 ways
- parallel access to labels and data
- block size 16 bytes
- FIFO replacement policy
- instruction size 32 bits

A 64 KB main memory, addressable at byte level, is also available. A counter and a ROM memory will be used to generate the instruction address trace that would be generated by the processor.

Description of the operation (see chronogram):

- If the requested address is available in cache memory, the **hit** signal is activated and the instruction is sent to the processor through the **inst** output. All this occurs in a single cycle.
- If the instruction is not available, the **hit** signal remains disabled and the **mem_en** signal is enabled to request the corresponding block from the main memory. All this occurs in the same cycle (0). The main memory operates synchronously. The requested block will be received through the **bus** (in four 32-bit words) in cycles 4, 6, 8 and 10. **mem_en** signal must remain active until the end of the whole process. The four words will be loaded into the four 32-bit registers. In cycle 11, the four words are written to the corresponding memory modules. In cycle 12, the instruction is referenced again, which must now yield a hit

cycle	0	0	1	2	3	4	5	6	7	8	9	10	11	12
@	@ ₁	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂	@ ₂
hit	1	0	0	0	0	0	0	0	0	0	0	0	0	1
mem_en	0	1	1	1	1	1	1	1	1	1	1	1	1	0
inst	inst ₁													inst ₂
bus						word ₀		word ₁		word ₂		word ₃		

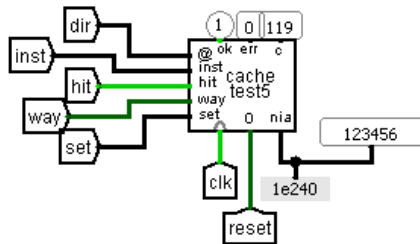
Design your cache with eight 32x32b RAM modules (data storage), two 32x8b RAM modules (tag and validity bit storage), one 32x1b RAM module (indicates the next lane to be replaced in each set), two 4:1x32 multiplexers, one 2:1x32 multiplexer, two 8-bit comparators and one decoder (to trigger writing in cycle 11 in the corresponding lane). See schematic on the front cover.

- Identify which bits of the address will be used to choose tag, set and word. Place the tag on the most significant bits of the address.
- Connect the components of the design.
- Draw the automaton of the cache control unit, so that it follows the described operation.

2.2 LABORATORY WORK: DESIGN WITH LOGISIM

- Create a new Logisim sheet and name it NIA-cache.circ.
- Load the library with the main memory component (Project / Load Library / Logisim Library / MP.circ) available in the Moodle *Logisim* folder. To facilitate the debugging of the cache memory we have programmed the main memory with dummy instructions. The first instruction is 0x000000000 and occupies the memory addresses (@0, @1, @2 and @3); the next instruction is 0x000000004 and occupies the addresses @4, @5, @6 and @7; the next 0x000000008 in @8, @9, @10 and @11; ...
- Place all components: counter, ROM, main memory (MP component), RAM modules, registers, comparators, multiplexers and decoder. Interconnect them according to your design
- Implement the cache memory control unit. You can make an implementation using a counter and a decoder, similar to the one used to control the main memory (see MP module design). Alternatively, you can resort to a traditional design: one register and two ROMs.
- Program the ROM with the address sequence you want to test.
- Debug your design until it works properly.
- Program the ROM with the sequence of addresses you get from the webdiis.unizar.es/~luisma/aoc2/nia4.php page from your NIA. Check that it works correctly.
- Before submitting the circuit you **must pass a test vector**. To do this:
 - Go to the Moodle resource "Submit NIA-cache", download the file "cache test5.circ" and save it in the same directory as your .circ file. Add it as an external component (Project / Load Library / Logisim Library / Select file "cache test5.circ").
 - Place a **test5 cache** component and connect its inputs to your circuit signals through tunnels.
 - Put a constant with your NIA in the **nia** input (you can type it in decimal and Logisim converts it to hexadecimal). Place a **probe** configured as "Unsigned decimal" on the same signal to check that the NIA is correct.
 - Place 3 **probes** on the **ok**, **err**, and **c** outputs of **cache test5**. Configure the **probes** of the **c** and **err** signals as "Unsigned decimal".
 - Activate the clock (Ctrl-K). If after 119 cycles the **ok** signal is activated, your circuit is working correctly. If at some point **c** stops counting it is because your circuit is faulty or the address sequence is not correct. In that case, the **err** signal will indicate an error code: 1) wrong instruction or address not aligned; 2) wrong track; 3) wrong set; 4) wrong hit/miss; 5) address repeated.

Commented [JLB1]: he puesto probe, no sé si acierto. El DeepL había puesto "a Ver component"



- i) When your circuit has passed the test vector you can submit it through the Moodle resource "NIA-cache delivery".